Producer-Consumer

ducer-Consumer · Bounded queue between encoder and uploader · Build with Wa

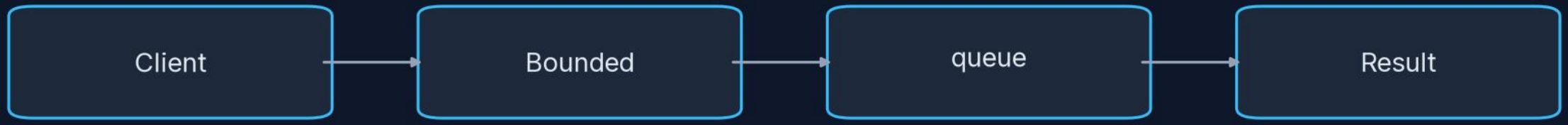
Today's Hook

- Today's force: Bounded queue between encoder and uploader
- We earn Producer-Consumer by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: BoundedQueue

The Problem

- Scenario: Bounded queue between encoder and uploader
- Constraints: solve «Bounded queue between encoder and uploader» with Producer-Consumer — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Bounded queue between encoder and uploader
- Success looks like: upload a

Structure



Producer-Consumer: Bounded queue between encoder and uploader

Real-World Scenario

- Running example: Bounded queue between encoder and uploader
- Key types: BoundedQueue
- Happy-path stdout cue: upload a
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```
package com.buildwithwaqar.patterns.concurrency.producerconsumer.problem;

import java.util.*;

public final class NaivePipe {
  public static void run(List<String> jobs){ for(String j: jobs) System.out.println("upload "+j); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Bounded queue between encoder and uploader
- Without Producer-Consumer, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.producerconsumer.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class BoundedQueue<T> {
  private final ArrayBlockingQueue<T> q;
  BoundedQueue(int cap){ q=new ArrayBlockingQueue<>(cap); }
  void put(T t) throws InterruptedException { q.put(t); }
  T take() throws InterruptedException { return q.take(); }
  int size(){ return q.size(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.concurrency.producerconsumer.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Producer-Consumer: encoder→uploader queue ===");
    System.out.println("--- naive ---"); NaivePipe.run(List.of("a","b"));
System.out.println("--- producer-consumer ---");
var q=new BoundedQueue<String>(2);
var done=new CountDownLatch(1); List<String> uploaded=new ArrayList<>();
Thread consumer=Thread.ofVirtual().start(() -> { try {
  for(int i=0;i<3;i++) uploaded.add(q.take());
  done.countDown();
} catch(InterruptedException e){ Thread.currentThread().interrupt(); }});
q.put("a"); q.put("b"); q.put("c"); done.await(2, TimeUnit.SECONDS);
System.out.println("uploaded="+uploaded);
  }
}
```

Walkthrough Output

```
=== Producer-Consumer: encoder→uploader queue ===
--- naive ---
upload a
upload b
--- producer-consumer ---
uploaded=[a, b, c]
```

Step-by-Step Trace

- Match stdout to stages — look for: upload a
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 35-producer-consumer ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Producer-Consumer
// Prefer records + sealed where they shrink the demo.
// Problem under test: Bounded queue between encoder and uploader
```

Trade-offs

- Use Producer-Consumer when «Bounded queue between encoder and uploader» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Producer-Consumer in product language.

Related Patterns

- Name the pattern usually confused with Producer-Consumer and separate the forces.
- Sketch the roles for «Bounded queue between encoder and uploader» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Bounded queue between encoder and uploader
- Naive pain → Producer-Consumer boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.